

LEMBAR KERJA PESERTA DIDIK

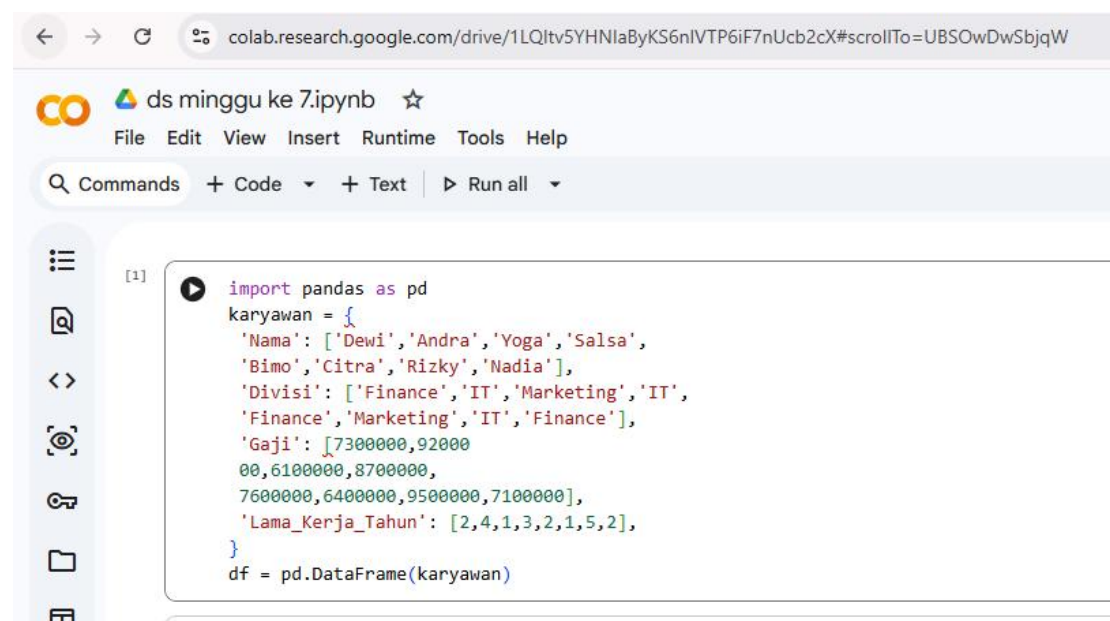
Studi Kasus: Eksplorasi & Seleksi Data dengan Pandal

Data Science

NAMA:	AZ ZAHRA ALVIATIN NUR ILMIAH
KELAS:	XII RPL 1
NO .ABSEN	24
TANGGAL:	31/08/20256

Studi Kasus: Data Karyawan Startup

DIBAWAH INI ADALAH DATANYA



The screenshot shows a Google Colab notebook interface. The browser address bar displays the URL: `colab.research.google.com/drive/1LQItv5YHNIaByKS6nIVTP6iF7nUcb2cX#scrollTo=UBSOwDwSbjqW`. The notebook title is "ds minggu ke 7.ipynb". The menu bar includes "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". Below the menu bar, there are tabs for "Commands", "+ Code", "+ Text", and a "Run all" button. On the left sidebar, there are icons for file management and search. The main code cell, labeled "[1]", contains the following Python code:

```
import pandas as pd
karyawan = {
    'Nama': ['Dewi', 'Andra', 'Yoga', 'Salsa',
            'Bimo', 'Citra', 'Rizky', 'Nadia'],
    'Divisi': ['Finance', 'IT', 'Marketing', 'IT',
             'Finance', 'Marketing', 'IT', 'Finance'],
    'Gaji': [7300000, 9200000, 8000000, 8700000,
            7600000, 6400000, 9500000, 7100000],
    'Lama_Kerja_Tahun': [2, 4, 1, 3, 2, 1, 5, 2],
}
df = pd.DataFrame(karyawan)
```

1. Tampilkan 4 baris pertama dari DataFrame `df` menggunakan fungsi yang sesuai.

KODE:



The screenshot shows a Google Colab notebook interface. The code cell, labeled "[2]", contains the following Python code:

```
df.head(4)
```

Below the code cell, there is a green checkmark icon and the text "0s", indicating that the code has been executed successfully.

OUTPUTNYA:

	Nama	Divisi	Gaji	Lama_Kerja_Tahun
0	Dewi	Finance	7300000	2
1	Andra	IT	9200000	4
2	Yoga	Marketing	6100000	1
3	Salsa	IT	8700000	3

2. Tampilkan shape, columns, dan info() dari DataFrame df.
KODENYA:

```
[3]
✓ Os
print(df.shape)
print(df.columns)
print(df.info())
```

OUTPUTNYA:

```
(8, 4)
Index(['Nama', 'Divisi', 'Gaji', 'Lama_Kerja_Tahun'], dtype='object')
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Nama            8 non-null     object
1   Divisi          8 non-null     object
2   Gaji            8 non-null     int64
3   Lama_Kerja_Tahun 8 non-null     int64
dtypes: int64(2), object(2)
memory usage: 388.0+ bytes
None
```

Pertanyaan Analisis: Dari hasil info() yang kamu dapatkan, apakah ada kolom yang datanya tidak lengkap (ada nilai kosong)? Bagaimana kamu bisa mengetahuinya dari output info() tersebut?

Jawab: Tidak ada kolom yang datanya kosong. Ini terlihat dari "Non-Null Count" di output info(), keempat kolom (Nama, Divisi, Gaji, Lama_Kerja_Tahun) semuanya menunjukkan angka 8 non-null, sama persis dengan total 8 baris data.

3. Gunakan loc untuk menampilkan kolom Nama dan Gaji pada baris berlabel 1 sampai KODENYA:

```
df.loc[1:4, ['Nama', 'Gaji']]
```

OUTPUTNYA:

	Nama	Gaji
1	Andra	9200000
2	Yoga	6100000
3	Salsa	8700000
4	Bimo	7600000

4. Gunakan `iloc` untuk menampilkan 3 baris pertama, hanya kolom `Divisi` dan `Lama_Kerja_Tahun`.

KODE:

```
df.iloc[0:3, [1, 3]]
```

OUTPUTNYA:

✓ 0s

df.iloc[0:3, [1, 3]]

	Divisi	Lama_Kerja_Tahun
0	Finance	2
1	IT	4
2	Marketing	1

Pertanyaan Analisis: Bandingkan hasil `df.loc[...]` pada nomor 3 dengan `df.iloc[...]` menggunakan angka yang sama (coba jalankan juga di Colab). Mengapa jumlah barisnya bisa berbeda, padahal angka yang ditulis sama?

DAN SAYA SUDAH MENCOBANYA JIKA NOMER 4 JADI `df.iloc[1:4]` outputnya seperti ini

[6]
✓ 1s

df.iloc[1:4]

	Nama	Divisi	Gaji	Lama_Kerja_Tahun
1	Andra	IT	9200000	4
2	Yoga	Marketing	6100000	1
3	Salsa	IT	8700000	3

JAWAB: Angkanya sudah saya tulis sama 1:4, tapi kok hasilnya beda jumlah baris karena `loc` sama `iloc` itu cara ngambilnya beda. `loc` itu ngambil berdasarkan nomor label yang nempel di tiap baris, dan dia ngambil SAMPAI HABIS ke angka terakhir yang ditulis, jadi pas nulis `loc[1:4]`, baris label 4 ikut keambil semua, makanya jadi 4 baris (1,2,3,4). Sedangkan `iloc` itu ngambil berdasarkan urutan posisi doang, kayak ngitung 0,1,2,3 gitu, dan dia BERHENTI SEBELUM angka terakhir — jadi pas nulis `iloc[1:4]`, yang keambil cuma posisi 1,2,3 aja, posisi 4-nya nggak ikut, karena aturan

iloc memang gitu: angka terakhir itu cuma jadi "batas berhenti", bukan ikut diambil. Makanya walaupun angkanya keliatan sama, iloc hasilnya lebih dikit satu baris dibanding loc.

5. Tampilkan karyawan divisi IT dengan Gaji di atas 8.500.000, menggunakan boolean indexing.

Kode:

```
df[(df['Divisi'] == 'IT') & (df['Gaji'] > 8500000)]
```

Outputnya:

	Nama	Divisi	Gaji	Lama_Kerja_Tahun
1	Andra	IT	9200000	4
3	Salsa	IT	8700000	3
6	Rizky	IT	9500000	5

6. Tampilkan karyawan dengan Lama_Kerja_Tahun kurang dari 2 ATAU Gaji di atas 9.000.000 (gunakan operator |)

Kodenya:

```
[8] df[(df['Lama_Kerja_Tahun'] < 2) | (df['Gaji'] > 9000000)]
```

Outputnya:

	Nama	Divisi	Gaji	Lama_Kerja_Tahun
1	Andra	IT	9200000	4
2	Yoga	Marketing	6100000	1
5	Citra	Marketing	6400000	1
6	Rizky	IT	9500000	5

7. Urutkan seluruh data karyawan berdasarkan kolom Gaji, dari yang tertinggi ke terendah

Kodenya:

```
df.sort_values(by='Gaji', ascending=False)
```

Outputnya:

	Nama	Divisi	Gaji	Lama_Kerja_Tahun
6	Rizky	IT	9500000	5
1	Andra	IT	9200000	4
3	Salsa	IT	8700000	3
4	Bimo	Finance	7600000	2
0	Dewi	Finance	7300000	2
7	Nadia	Finance	7100000	2
5	Citra	Marketing	6400000	1
2	Yoga	Marketing	6100000	1

8.Pertanyaan Reflektif: Di dunia kerja nyata (HR analyst / data analyst), kapan boolean indexing seperti pada nomor 5 dan 6 biasanya digunakan? Berikan satu contoh situasi nyata DI LUAR kasus data karyawan pada LKPD ini

Jawaban: Boolean indexing biasa dipakai pas mau nyaring data yang banyak banget berdasarkan syarat tertentu, tanpa cek satu-satu manual. Contohnya di toko online, tim data analyst bisa nyaring pelanggan yang belanjanya di atas 1 juta sebulan DAN belum pernah retur barang, buat dikirimin promo khusus. Cara ini jauh lebih cepet dibanding ngecek data manual di excel.

F. Praktik Mandiri / Kelompok (± 30 menit)

Bagi siswa ke dalam kelompok kecil (3–4 orang). Berikan dataset sederhana berisi minimal 8 baris data (misalnya data siswa, data produk, atau data transaksi kantin sekolah). Minta setiap kelompok:

1. Menampilkan karakteristik dataset menggunakan `head()`, `info()`, dan `describe()`

SAYA PAKAI DATA SISWA

```
[2]
✓ Os  import pandas as pd

siswa = {
    'Nama': ['Rafi', 'Dini', 'Farhan', 'Lita',
             'Galih', 'Mira', 'Adit', 'Nayla'],
    'Kelas': ['XII RPL 1', 'XII RPL 1', 'XII RPL 2', 'XII RPL 2',
              'XII RPL 1', 'XII RPL 2', 'XII RPL 1', 'XII RPL 2'],
    'Nilai_Matematika': [85, 70, 92, 60, 78, 88, 55, 95],
    'Nilai_IPA': [80, 75, 85, 65, 70, 90, 60, 88],
}

df2 = pd.DataFrame(siswa)
```

Dibawah ini saya menampilkan karakteristik dataset menggunakan `head()`, `info()`, dan `describe()`

```
[3]
✓ Os  df2.head()
df2.info()
df2.describe()
```

*** <class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	Nama	8 non-null	object
1	Kelas	8 non-null	object
2	Nilai_Matematika	8 non-null	int64
3	Nilai_IPA	8 non-null	int64

dtypes: int64(2), object(2)
memory usage: 388.0+ bytes

	Nilai_Matematika	Nilai_IPA
count	8.000000	8.000000
mean	77.875000	76.62500
std	14.884676	11.00568
min	55.000000	60.00000
25%	67.500000	68.75000
50%	81.500000	77.50000
75%	89.000000	85.75000
max	95.000000	90.00000

2 .Melakukan minimal satu seleksi data menggunakan `loc` dan satu seleksi menggunakan `iloc`.

Dibawah ini saya seleksi data menggunakan `loc`

```
[4] df2.loc[0:3, ['Nama', 'Nilai_Matematika']]
```

	Nama	Nilai_Matematika
0	Rafi	85
1	Dini	70
2	Farhan	92
3	Lita	60

Dibawah ini saya seleksi data menggunakan iloc

```
[5] df2.iloc[0:3, [0, 2]]
```

	Nama	Nilai_Matematika
0	Rafi	85
1	Dini	70
2	Farhan	92

3 .Membuat satu filter dengan dua kondisi sekaligus (menggunakan & atau |).

```
df2[(df2['Kelas'] == 'XII RPL 1') & (df2['Nilai_Matematika'] > 75)]
```

	Nama	Kelas	Nilai_Matematika	Nilai_IPA
0	Rafi	XII RPL 1	85	80
4	Galih	XII RPL 1	78	70

4 .Mengurutkan hasil filter tersebut berdasarkan salah satu kolom numerik, lalu mempresentasikan temuannya secara singkat ke kelas.

```
hasil_filter = df2[(df2['Kelas'] == 'XII RPL 1') & (df2['Nilai_Matematika'] > 75)]  
hasil_filter.sort_values(by='Nilai_Matematika', ascending=False)
```

	Nama	Kelas	Nilai_Matematika	Nilai_IPA
0	Rafi	XII RPL 1	85	80
4	Galih	XII RPL 1	78	70

Kita filter siswa kelas XII RPL 1 yang nilai matematikanya di atas 75, terus diurutin dari tertinggi. Hasilnya cuma ada 2 orang yang lolos: Rafi (85) sama Galih (78), dan Rafi yang nilainya paling tinggi. Ini nunjukin boolean indexing + sort_values bisa langsung nyaring dan ngurutin data tanpa harus cek satu-satu manual.

G. Asesmen Formatif

Beberapa contoh soal singkat untuk mengecek pemahaman peserta didik di akhir pertemuan:

1. Jelaskan perbedaan mendasar antara loc dan iloc, beserta contoh kasus kapan masing-masing sebaiknya digunakan.

Jawab: Jadi loc sama iloc itu sama-sama buat ngambil data, cuma caranya beda. Loc manggil pake nama/label-nya langsung, kayak misal mau ambil baris yang labelnya "3" atau kolom yang namanya "Gaji". Iloc manggilnya pake nomor urut doang, kayak "pokoknya baris ke-0, ke-1" gitu, gak peduli namanya apa.

Contohnya kasus : kalo lagi nyari data karyawan berdasarkan nama kolom yang udah kita tau, kayak mau liat kolom "Nama" sama "Gaji" doang, enakan pake loc soalnya tinggal sebut nama kolomnya langsung. Tapi kalo misal cuma mau "ambil 5 data teratas buat dicek sekilas" tanpa peduli itu kolom apa aja, ya mending pake iloc, tinggal `df.iloc[0:5]` aja beres.

2. Diberikan DataFrame df dengan kolom 'Umur' dan 'Kota', tuliskan kode untuk menampilkan baris dengan Umur di atas 20 DAN Kota bernilai 'Surabaya'.

Jawaban: `df[(df['Umur'] > 20) & (df['Kota'] == 'Surabaya')]`

3. Mengapa boolean indexing lebih disarankan dibanding perulangan (for loop) manual untuk memfilter data pada DataFrame berukuran besar?

Jawab: Soalnya boolean indexing itu ngecek semua baris bareng-bareng sekaligus, jadi lebih cepet daripada for loop yang harus ngecek satu-satu. Kalo datanya udah ribuan baris, for loop bakal lemot parah, sedangkan boolean indexing masih ngebut soalnya diproses pake NumPy di belakangnya.

4. Tuliskan kode untuk mengurutkan DataFrame df berdasarkan kolom 'Nilai' dari yang tertinggi ke terendah

Jawab: `df.sort_values(by='Nilai', ascending=False)`